

Localized Open-Source LLM Aware Retrieval Augmented Generation of Legal Documents: A Case Study on Indian Constitution and Penal Code

Pratiksha Phukon

Department of Computer Application
Sikkim University
Sikkim 737102, India
pratixaphukon@gmail.com

Yogesh Lokhar

Department of Computer Application
Sikkim University
Sikkim 737102, India
yogeshlokhar@gmail.com

Partha Pratim Ray

Department of Computer Application
Sikkim University
Sikkim 737102, India
ppray@cus.ac.in

Abstract—Large Language Models (LLMs) are advanced artificial intelligence systems which are designed to generate human-like language based on their training data. Retrieval-Augmented Generation (RAG) is a framework that enhances the response of LLM by incorporating external information sources, leading to more accurate, context-relevant responses. However, LLMs commonly struggle with contextual relevance, especially in particular fields such as legal documentation, where specific local laws are crucial. Furthermore, there is limited research on the use of RAG technology in the legal area, particularly for producing and analyzing legal documents. This study develops a chatbot using RAG to answer questions on the Indian Constitution (IC) and the Indian Penal Code (IPC), with a Streamlit-based interface to display legal queries and their generated responses. This study analyzes the evaluation of various LLMs integrated with RAG including (including (i) Llama3 8b, (ii) Mistral 7b, (iii) Gemma2 7b, (iv) Phi3 3.8b, and (v) Qwen2 7b and it evaluates their performance in terms of relevance, faithfulness, context recall, and context precision.

Keywords—Large Language Model, Retrieval-Augmented Generation, Localized Interface, Question-Answer Visualizer, Legal Documents.

I. INTRODUCTION

The LLMs are advanced artificial intelligence systems that are designed to generate human-like language based on the data on which they have been trained. Conventional LLMs, which are non-localized and cloud-based, pose security problems, network dependence, and cost. Training and maintaining LLMs can be costly and time-consuming, necessitating extensive computational resources [1]. To address these limitations, implementing localized, open-source LLMs emerges as a viable solution providing improved security and cost savings [2].

Though these models are trained from diverse sources with large amounts of data, they have limitations, particularly in specialized fields where they produce hallucinations [3] (information that goes beyond their training data). To address these limitations, RAG is used to enhance the ability of large language models to retrieve relevant information from a knowledge base by utilizing semantic similarity calculations. This approach improves both the quality and relevance of the generated content by incorporating a retrieval mechanism delivering accurate contextual information. In RAG, a language model is paired with retriever components that access vast knowledge bases, enabling it to perform tasks

such as question answering, summarization, and text generation. This allows the system to produce responses that are more precise, informative, and relevant[4].

The RAG is an advanced technique that enhances the LLM by integrating its retrieval capabilities to retrieve relevant content. In the rag process, first and foremost it does data collection where it gathers relevant documents to create a comprehensive dataset then it encodes the documents and after that it does indexing where the encoded document split into chunks to manage and process large volumes of data effectively. Each chunk can be independently tagged, indexed, and analyzed this improves the accuracy and efficiency of subsequent analytical processes and facilitates the retrieval of sections or topics when needed. The process of encoding which transforms textual chunks into numerical representations, or vectors using an embeddings model. Encoding is essential for advanced text analysis and retrieval tasks. Vectors encapsulate relationships and similarities between different pieces of text for more accurate retrieval. After encoding the vector, it is stored in the vector database. The query's relevant response is then retrieved from the database, combined with LLM, and shown to the user as a prompt [5].

This paper explore the application of RAG in the legal domain, specifically for IPC and IC documents. Due to the limited research on RAG-involved techniques on the IPC and IC, this study aims to provide a system that efficiently retrieves accurate and up-to-date legal information. The key contributions of the research are:

1) *QA Chatbot Visualization*: A QA chatbot visualization system is developed using Streamlit. This system combines LLMs with RAG to make it easier to access legal information from the IC and IPC.

2) *Comprehensive LLM Evaluation*: A detailed comparison of five different LLMs such as (i) Llama3, (ii) Mistral, (iii) Gemma2, (iv) Phi3, and (v) Qwen2 is carried out based on total duration the overall time spent on generating the response, prompt eval count is the number of tokens when the prompt is generated, prompt eval duration time which is measured in nanosecond for analyzing the prompt, eval count number of tokens in the reply, eval duration time which is measure in nanosecond to generating the reply, tokens per second is the number of words generated in seconds in the

response and relevance, faithfulness, context recall, context precision.

3) *Performance Comparison Metrics*: A detailed analysis is included of comparison metrics regarding average scores, highest scores, lowest scores, performance rankings, and total performance rankings. This comparison across multiple fields provides a clearer understanding of which LLM performs best in specific areas.

This paper is organized as follows. Section II presents the related works. Section III outlines the proposed methodology for developing the QA chatbot visualization system. Section IV presents the results and discusses the performance metrics of the evaluated LLMs. At last, section V concludes the paper with a summary of the findings.

II. RELATED WORK

The RAG has significantly emerged as a crucial advancement in the LLMs, which has limitations such as hallucinations and outdated information [6][7]. RAG combines retrieval systems with generative models, which allows LLMs to access external information. RAG was introduced in 2020 and has proven useful in reducing inaccuracies when LLMs interact with information beyond their pre-trained knowledge. [8] and evolved from naive retrieval to advanced techniques and modular systems.

This system advances in various multimodal applications, by integrating a variety of data formats such as images, code, audio, and all text document format[9][10]. This improved LLMs performance in visual question answering. The content which local models generated, RAG is essential for maintaining or retrieving updated knowledge and incorporating rare information. For open-domain question answering, Generated-Augmented Retrieval has shown better results by augmenting queries with generated contexts [11].

The main challenge which system encounters is information overload during retrieval [12][13]. To solve this issue, researchers developed evaluation methods for re-ranking and filtering [14][15]. Evaluation methods have also evolved, with frameworks such as Retrieval-Augmented Generation Assessment RAGAS offering multidimensional assessments. The development of advanced LLMs like as GPT-4 and Llama continues to have an impact on RAG research, potentially improving performance and expanding capabilities [16][17].

III. PROPOSED METHODOLOGY

This section presents the underlying methodology behind the study. Explaining how RAG operates and detailing the tools used to develop RAG using LangChain, Ollama which was used to pull various LLMs, Chroma DB to store the chunks which is converted into vectors, Streamlit is used for interface to display the the query and generated output, Python 3.9, Intel Xeon(R) Processor, 1 Terabyte Hard drive, Quadro RTX 5000 etc. This procedure generally comprises three major factors: (i) indexing and (ii) retrieval, and (iii) generation.

A. Indexing

1) *Loading*: The process starts by loading the IPC and IC data in PDF form. Document Loader is used to load data in the system.

2) *Splitting*: The loaded documents are encoded and divided into parts using Text Splitters. The loaded document get encoded and separated into small and manageable chunks, which helps to manage the vast amount of documents. Each chunk is indexed, tagged, and analyzed separately which makes the retrieval of specific sections easy.

3) *Storing*: The manageable chunks are turned into vectors or numerical representations, this process is called encoding. This procedure is necessary for advanced text analysis and retrieval tasks. Nomic-embed-text is used, which capture the semantic meaning of the text. Vector encapsulated the relationship and similarities between different pieces of text which enables more accurate retrieval. Then stores the vector into the vector database.

Algorithm: IPC-IC QA LLM-RAG Chatbot

Input: User query

Output: Chatbot's response

Initialize:

Import required libraries (Streamlit, LangChain components, standard libraries)

Define `get_embedding_function()` to load the Ollama embeddings using `OllamaEmbeddings`.

Cache `get_llm()` and `get_qa_chain()` for efficiency

Load and Process Documents:

- Define `load_and_process_documents()` to:
 - Load all PDF documents from a directory using `DirectoryLoader`.
 - Split the documents into chunks using `CharacterTextSplitter`.
- Cache the document loader for performance.

Initialize the Vector Store:

- Define `get_vectorstore()` to:
 - Load embeddings via `get_embedding_function()`.
 - Load and split documents via `load_and_process_documents()`.
 - Set up the Chroma vector store with document embeddings.
 - Configure an SQL-based `RecordManager` to handle indexing of the documents.
 - Index the processed documents incrementally.

Configure Metrics Logging:

- Calculate `_and_log_metrics()`:
 - Log and track processing times for `queries.load_and_process_documents()`.

Configure User Interface:

Apply custom CSS styling, Set app title and sidebar options and Initialize chat history in session state.

While (True):

Display previous messages from chat history and Capture user input query.

If query exists, Then Do

- Display and save user's message.
 - Process the query:
 - Record timestamps.
 - Retrieve `qa_chain()` and obtain result.
 - Log metrics asynchronously.
 - Display and save assistant's response.
-

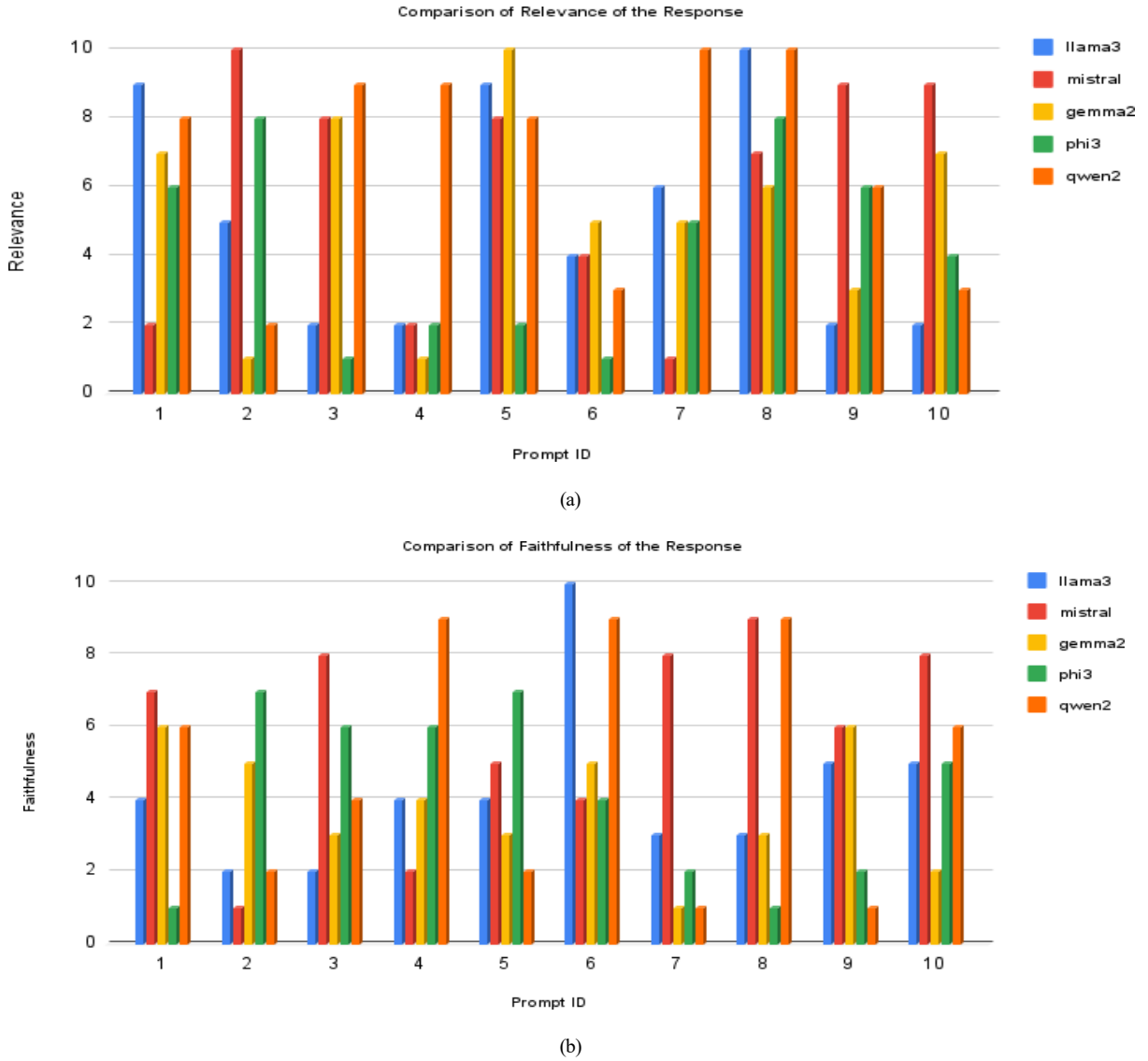


Fig. 1. (a) Relevance of the responses generated by different models, (b) Faithfulness of the responses generated by different models.

The generated vectors are stored in Chroma DB, a vector store designed to handle high-dimensional vectors. This ensures that the data is available and can be used in the future. Vector storage provides similarity measures, which are used to retrieve relevant vectors quickly and accurately based on semantic similarities.

B. Retrieval

When a user submits a question, the system preprocesses it in the same manner as the indexed documents. Tokenization is used to break down the question into distinct subwords, which is then followed by stopword elimination. The question is then encoded as a vector and numerically represented using embedding techniques. Using semantic similarity measures, the system determines how closely the question vector matches the vector-stored chunks in the database and retrieves the most relevant chunks and selects them based on similarity scores to focus the information.

C. Generate

After receiving all the relevant information, text generation occurs. The query is encoded as a vector, and the most relevant chunks from the vector database are delivered into the LLM. The LLM responds with contextually relevant information based on its training data and retrieved chunks. The prompt created for the LLM includes both the original question and the received chunks. Finally, the generated response is delivered to the user or application.

IV. RESULTS AND DISCUSSION

A total of 10 queries were employed to evaluate results across various fields, with each query assigned a unique prompt ID, as shown in Table I. This structured approach helps to clearly analyze the performance of the different LLM models tested in this study. The queries cover various aspects

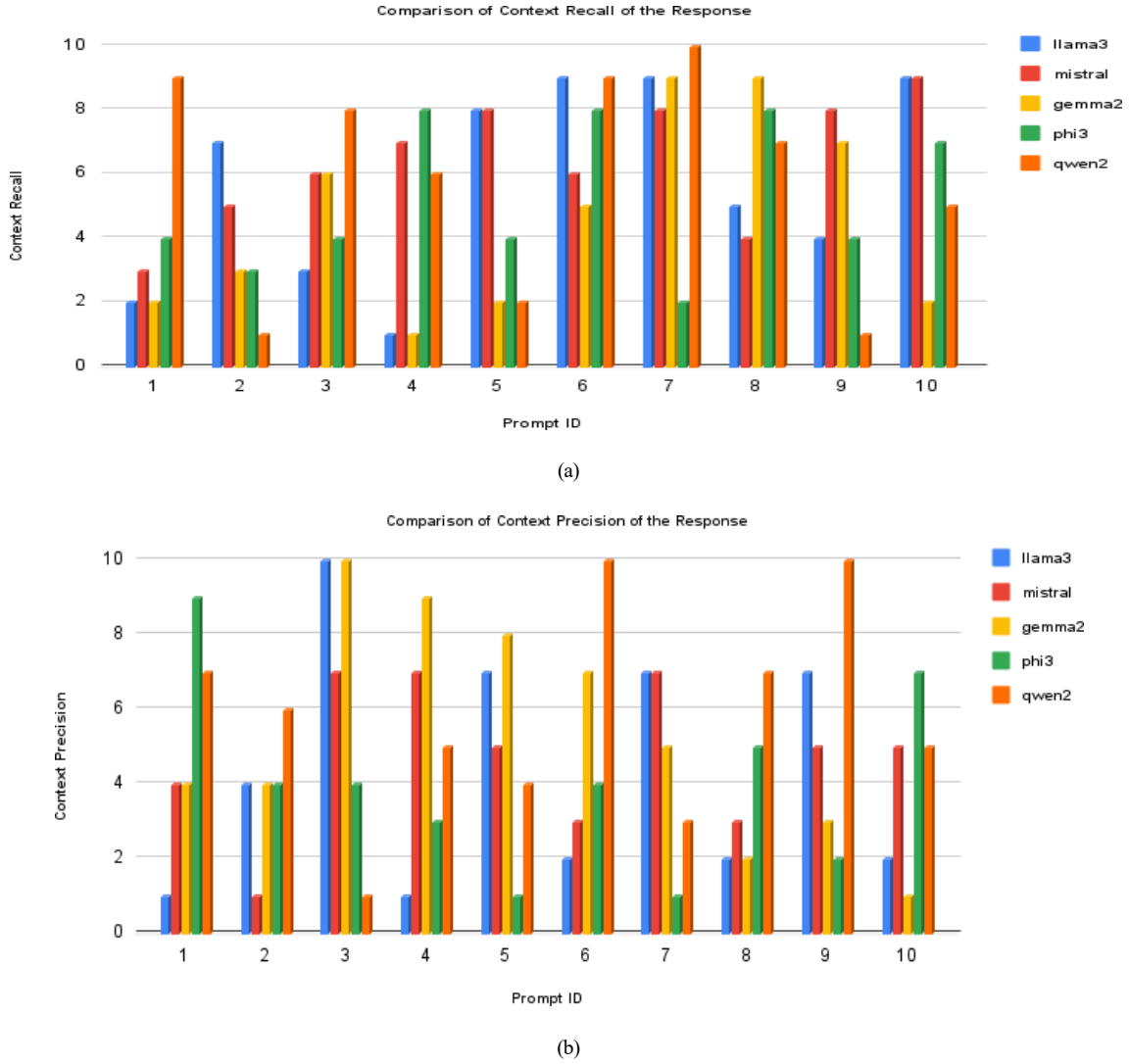


Fig. 2. (a) Context recall of the responses generated by different models, (b) context precision of the responses generated by different models.

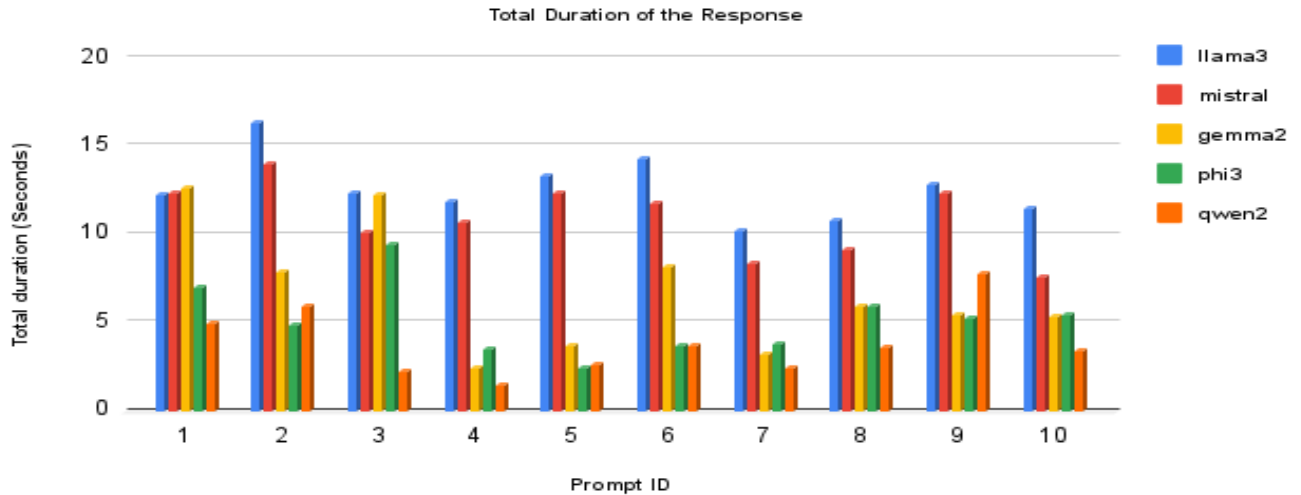
of legal information, for thorough comparisons of the models abilities.

TABLE I. TEST PROMPTS

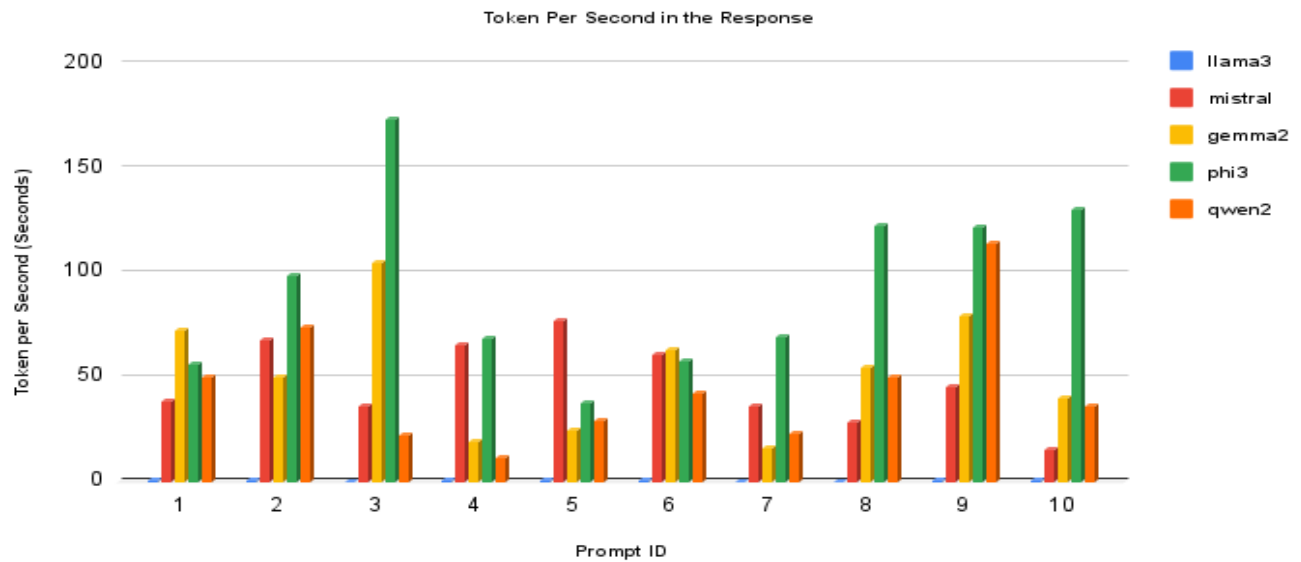
Prompt ID	Queries
1	What is the difference between culpable homicide and murder?
2	What are the specific offenses and penalties related to counterfeiting currency notes and government stamps?
3	What constitutes criminal conspiracy
4	What is the punishment for murder?
5	How is forgery defined?
6	What are the duties of the President of India?
7	How is the Governor's executive power defined in the Constitution?
8	What is the punishment for theft?
9	How do States borrow money according to the Constitution?
10	What are the restrictions on trade and commerce between States?

Fig. 1. displays the (a) relevance and (b) faithfulness of the responses generated by different models, calculated using the RAGAs metrics. RAGAs is an open-source framework for evaluating RAG systems by quantifying their performance. We evaluated our system on a scale of 1 to 10, focusing on relevance, faithfulness, context recall, and context precision. Llama3 exhibits the lowest scores for both relevance and faithfulness, indicating challenges in addressing the queries and aligning with the expected information. In contrast, Mistral and Gemma2 demonstrate higher relevance and faithfulness scores across various prompts, reflecting their effectiveness in generating accurate and pertinent responses. Overall, this comparison highlights the strengths and weaknesses of each model in terms of relevance and faithfulness. Fig. 2. displays the context recall and context precision of the responses generated by different models. In this assessment, all LLMs demonstrate similar performance in context recall across various queries, indicating a consistent ability to recall relevant information.

However, Llama3 shows the lowest context precision, while the other models have performed similarly across different queries. Table II is the comparison metrics which shows the average of five LLMs where it is compared in various fields such as relevance, faithfulness, context recall, context precision. Where mistral and qwen2 has performed better than other LLM models.



(a)



(b)

Fig. 3. (a) Total time taken by different models to generate responses, (b) the rate of tokens produced per second.

TABLE II. COMPARISON METRICS OF AVERAGE SCORES OF LLMs

Llm	Relevance	Faithfulness	Context Recall	Context Precision
LLama3	5.10	3.70	4.10	5.30
Mistral	5.30	5.30	6.10	4.30
Gemma2	5.10	3.70	4.10	5.30
Phi3	4.30	4.10	5.20	4.00
Qwen2	6.80	4.90	5.70	5.80

Table III presents the highest score metrics, showing the top-performing LLM models across various fields such as relevance, faithfulness, context recall, and context precision. In this comparison, Qwen2 achieved the highest scores in relevance and context precision, while Mistral secured the highest scores in both faithfulness and context precision.

TABLE III. HIGHEST SCORES METRICS

Metric	Highest Score	LLM
Relevance	6.80	Qwen2
Faithfulness	5.30	Mistral
Context Recall	6.10	Mistral
Context Precision	5.80	Qwen2

TABLE IV. LOWEST SCORE METRICS

Metric	Lowest Score	LLM
Relevance	4.30	Phi3
Faithfulness	3.70	Gemma2
Context Recall	4.10	Gemma2
Context Precision	4.00	Phi3

Table IV presents the lowest score metrics, indicating the LLM models with the lowest performance in fields such as relevance, faithfulness, context recall, and context precision. According to this table, Phi3 has the lowest scores in relevance and context precision, while Gemma2 shows the lowest scores in faithfulness and context recall. Table V presents the comparison metrics of overall performance ranking where all models are ranked based on their performance and their average scores are also over there which average of various fields of comparison metrics such as relevance, faithfulness, context recall, context precision.

TABLE V. OVERALL PERFORMANCE RANKING

Rank	LLM	Average Score
1	Qwen2	5.80
2	Mistral	5.25
3	Llama3	4.83
4	Gemma2	4.55
5	Phi3	4.40

Table VI shows a comparison metrics which explain the detail performance of LLMs based on ranking with various field of comparison metrics. Fig. 3. shows the total time taken by different models to generate responses and the rate of tokens produced per second. The time is calculated using (1):

$$Duration = (ET - ST) \times 10^9 \quad (1)$$

where ET is the end time and ST is the start time. This data indicates that Llama3 took the longest time to respond, while Qwen2 was the quickest. Additionally, the number of tokens generated per second is calculated using (2):

$$Tokes\ per\ sec = \frac{EC}{ED} \times 10^9 \quad (2)$$

if $eval_duration > 0$ else 0

where EC is the evaluation count and ED is the evaluation duration (noting that this calculation only applies if the evaluation duration is greater than zero). According to this analysis, Phi3 generated the most tokens per second, while Llama3 produced the least.

TABLE VI. COMPARISON METRICS OF LLM PERFORMANCE

Rank	Relevance	Faithfulness	Context Recall	Context Precision
1	Qwen2 (6.80)	Mistral (5.30)	Mistral (6.10)	Qwen2 (5.80)
2	Mistral (5.30)	Qwen2 (4.90)	Qwen2 (5.70)	Gemma2 (5.30)
3	LLama3 (5.10)	Phi3 (4.10)	LLama3 (5.70)	Mistral (4.30)
4	Gemma2 (5.10)	LLama3 (4.20)	Phi3 (5.20)	LLama3 (4.30)
5	Phi3 (4.30)	Gemma2 (3.70)	Gemma2 (4.10)	Phi3 (4.00)

V. CONCLUSION

This study concludes with the development of a RAG powered chatbot utilizing various LLMs for providing accurate legal query responses related to the IC and IPC. The implementation of a Streamlit interface enables efficient retrieval of relevant legal information, and visualization of generated responses. A comparative analysis highlights the performance of various LLMs in terms of relevance, faithfulness, context recall, and precision. Future work aims to enhance legal query resolution and overall performance.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459-9474, 2020.
- [2] G. Izacard and E. Grave, "Efficient retrieval-augmented generation from unstructured text," *arXiv:2112.04426*, 2021.
- [3] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, et al., "Siren's song in the AI ocean: A survey on hallucination in large language models," *arXiv preprint arXiv:2309.01219*, 2023.
- [4] K. Shuster, S. Roller, M. Weston, and J. Kiela, "The RAG handbook: Using retrieval-augmented generation in NLP," *FreeCodeCamp*, 2023.
- [5] P. Lewis, J. Hoffmann, S. Petroni, and A. Joulin, "Towards efficient retrieval-augmented generation at scale," *arXiv:2104.05140*, 2021.
- [6] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, Dec. 2023.
- [7] M. U. Hadi, R. Qureshi, A. Shah, M. Irfan, A. Zafar, M. B. Shaikh, N. Akhtar, J. Wu, S. Mirjalili, "A survey on large language models: Applications, challenges, limitations, and practical usage," *Authorea Preprints*, Oct. 2023.
- [8] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain, and J. Gao, "Large language models: A survey," *arXiv preprint arXiv:2402.06196*, Feb. 2024.
- [9] P. Zhao, H. Zhang, Q. Yu, Z. Wang, Y. Geng, F. Fu, L. Yang, W. Zhang, and B. Cui, "Retrieval-augmented generation for AI-generated content: A survey," *arXiv preprint arXiv:2402.19473*, Feb. 2024.
- [10] R. Zhao, H. Chen, W. Wang, F. Jiao, X. L. Do, C. Qin, B. Ding, X. Guo, M. Li, X. Li, and S. Joty, "Retrieving multimodal information for augmented generation: A survey," *arXiv preprint arXiv:2303.10868*, Mar. 2023.
- [11] Y. Mao, P. He, X. Liu, Y. Shen, J. Gao, J. Han, and W. Chen, "Generation-augmented retrieval for open-domain question answering," *arXiv preprint arXiv:2009.08553*, Sep. 2020.
- [12] Y. Huang and J. Huang, "A survey on retrieval-augmented text generation for large language models," *arXiv preprint arXiv:2404.10981*, Apr. 2024.
- [13] N. Kandpal, H. Deng, A. Roberts, E. Wallace, and C. Raffel, "Large language models struggle to learn long-tail knowledge," in *Int. Conf. Mach. Learn.*, pp. 15696-15707, Jul. 2023.
- [14] N. Wiratunga, R. Abeyratne, L. Jayawardena, K. Martin, S. Massie, I. Nkisi-Orji, R. Weerasinghe, A. Liret, and B. Fleisch, "CBR-RAG: case-based reasoning for retrieval augmented generation in LLMs for legal question answering," in **Proc. Int. Conf. Case-Based Reasoning**, Cham, Switzerland: Springer Nature, 2024, pp. 445-460.
- [15] A. Louis, G. van Dijk, and G. Spanakis, "Interpretable long-form legal question answering with retrieval-augmented large language models," in **Proc. AAAI Conf. Artif. Intell.**, vol. 38, no. 20, pp. 22266-22275, 2024.
- [16] J. Chen, H. Lin, X. Han, and L. Sun, "Benchmarking large language models in retrieval-augmented generation," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 16, pp. 17754-17762, 2024.
- [17] I. Radeva, I. Popchev, L. Doukovska, and M. Dimitrova, "Web Application for Retrieval-Augmented Generation: Implementation and Testing," *Electronics*, vol. 13, no. 7, p. 1361, 2024.